

LC 471: Encode String with Shortest Length

© Authored by: Arpit Raj, LNMIIT Jaipur | Date: September 4, 2026

Encoding Format

The goal is to find the shortest encoded string. The encoding format is based on finding repeated patterns: $k[\text{encoded_string}]$

- $\text{aaaaaa} = 6[\text{a}]$
- $\text{ababab} = 3[\text{ab}]$

Dynamic Programming State & Options

Let $\text{dp}[l][r]$ = shortest encoded string for the substring $s[l \dots r]$.

For any interval $s[l \dots r]$, there are 2 main options to consider:

Option 1: Split

Split the string into two independent parts $[l \dots k]$ and $[k+1 \dots r]$ and solve them independently.

Example: ababcd can be split into abab and cd .

Formula: $\text{dp}[l][k] + \text{dp}[k+1][r]$

Option 2: Repeat / Encode

Encode the entire interval as a repetition.

Check if $s[l \dots r]$ can be represented as $(k \times \text{smaller substring})$.

Example:

$s[l \dots r] = \text{"ababab"} \rightarrow \text{"ab"} + \text{"ab"} + \text{"ab"}$

Here, length = 6, $\text{ab} = 2$, so $k = (6/2) = 3$. Hence $3[\text{ab}]$.

Finding the Pattern & Crucial Observations

Let interval = `s[1...r]` , and `len = (r - 1 + 1)` .

Now try every possible pattern length: `p = 1 ..... len / 2` where `len % p == 0` .

Important Example: `abababab`

Initially, it might look like it should be encoded as `2[abab]` .

However, `abab` can itself be encoded optimally as `2[ab]` .

Hence the final, most optimal answer is `2[2[ab]]` .

Conclusion: Don't use a raw substring when repeating, because the substring itself is a DP state that might have a shorter representation.

Basically, when encoding the pattern, the structure looks like:

```
to_string(repeatcount) + "[" + dp[l][l+p-1] + "]"
```

Where the inner content is also derived from the DP table to ensure shortest length.

C++ Implementation

```
class Solution {
public:
    string encode(string s) {
        int n=s.size();

        vector<vector<string>> dp(
            n,
            vector<string>(n,"")
        );

        for(int len=1;len<=n;len++){
            for(int l=0;l+len-1<n;l++){
                int r=l+len-1;

                // initially take the string as it is
                dp[l][r]=s.substr(l,len);

                // opt1: split
                // [l.....k][k+1.....r]
                for(int k=l;k<r;k++){
                    string curr=dp[l][k]+dp[k+1][r];
                    if(curr.size()<dp[l][r].size()){
                        dp[l][r]=curr;
                    }
                }

                // opt2: encode by checking if s[l...r] is made of
                // smaller substrings
                for(int p=1;p<=len/2;p++){

                    if(len%p!=0) continue;
                    bool same=true;

                    string pattern=s.substr(l,p);

                    for(int i=l+p;i<=r;i+=p){
                        if(s.substr(i,p)!=pattern){
                            same=false;
                            break;
                        }
                    }
                }
            }
        }
    }
};
```

```

        if(same){
            int times=len/p;

            string curr=
                to_string(times)
                +"["
                +dp[l][l+p-1]
                +"]";

            if(curr.size()<dp[l][r].size()){
                dp[l][r]=curr;
            }
        }
    }
}

return dp[0][n-1];
}

};

```